



События

Преподаватель:

канд. техн. наук, доц. Озерова Г.П.

События

Событие – это момент времени, в который может быть совершено некоторое действие.

На HTML-странице могут быть сгенерированы следующие события:

- события, связанные с мышью;
- события, связанные с клавиатурой;
- события, связанные с загрузкой страницы;

Обработчики событий

При наступлении любого события может быть запущена функция, называемая **обработчиком событий**.

В обработке практически всех событий доступен объект **event**, который содержит множество свойств, описывающих данное событие, например координаты курсора мыши, нажатая клавиша и пр.

События, связанные с мышью

Событие и описание	Особенности
onmousedown - кнопка мыши нажата над элементом.	Эти события предоставляют текущие координаты курсора в двух видах: - относительно окна (левого верхнего угла): event.clientX, event.clientY - относительно документа (левого верхнего угла, с учетом невидимой части, если документ был прокручен): event.pageX, event.pageY
onmouseup - кнопка мыши отпущена над элементом.	
onmouseover - мышь появилась над элементом.	
onmouseout - мышь ушла с элемента.	
onmousemove - каждое движение мыши над элементом генерирует это событие.	
onclick - вызывается при клике мышью, то есть при onmousedown , а затем onmouseup на одном элементе через короткий промежуток времени	При клике (а также других событиях, связанных с мышью) можно определить, какая кнопка нажата: event.which = 1 – левая кнопка event.which = 2 – средняя кнопка event.which = 3 – правая кнопка
oncontextmenu - вызывается при клике правой кнопкой мыши на элементе.	Отменить вывод контекстного меню: <code><button oncontextmenu="return false"> Правый клик </button></code>
ondblclick - вызывается при двойном клике по элементу.	

События, связанные с мышью

Задание 1. Указать последовательность событий, которая позволяет нарисовать прямоугольник в графическом редакторе или выделить объект.

События, связанные с клавиатурой

Событие и описание	Особенности
onkeydown - срабатывает, когда пользователь нажимает клавишу	Для того, чтобы узнать, какая именно клавиша нажата или отпущена, используется объект event : event . keyCode
onkeyup - срабатывает, когда пользователь отпускает клавишу	
onkeypress - срабатывает, когда пользователь нажимает символьную клавишу, управляющие клавиши это событие не генерируют	

События, связанные с загрузкой

Событие и описание	Особенности
onDOMContentLoaded - генерируется, когда браузер полностью загрузил HTML и построил DOM-дерево	Означает, что все DOM-элементы разметки уже созданы, можно их искать, вешать обработчики, создавать интерфейс, но при этом, возможно, еще не догрузились какие-то картинки или стили.
onload - генерируется, когда браузер загрузил все ресурсы	Страница и все ресурсы загружены.
onbeforeunload/ onunload - генерируется при уходе со страницы	Можно проверить, сохранил ли посетитель изменения, уточнить, действительно ли он хочет покинуть страницу.

События, связанные с формой

Событие	Описание	Особенности
onclick	вызывается при клике мышью по элементу формы	
onchange	Значение было изменено.	Для текстовых полей срабатывает при потере фокуса.
oninput	Срабатывает при каждом изменении значения.	
oncut/ oncopy/ onpaste	Действия по вырезанию/копированию/вставке.	Свойство event.clipboardData предоставляет доступ на чтение/запись в буфер обмена
onblur	возникает при потере элемента фокуса	
onfocus	возникает при получении фокуса элементом	

События, связанные с формой

Задание 2. Указать последовательность событий, которая происходит, когда пользователь вводит значение в текстовое поле.

Обработчики событий

Обработчик событий может быть привязан к элементу:

- через атрибуты HTML;
- через DOM свойства элемента;
- метода **`addEventListener`**.

Назначение событий

События могут быть включены в тег HTML в качестве атрибутов:

`<тег событие="обработчик события" ...>`

Выполняется данная конструкция следующим образом:

- если над тегом срабатывает событие, описанное в качестве атрибута,
- то выполняется соответствующий обработчик событий.

С помощью событий, описанных в теге, можно реализовать динамическое изменение атрибутов тегов, как для текущего элемента, так и для любого другого.

Назначение событий

Пример. Изменим одну картинку на другую при наведении на нее мышкой, а при снятии мыши с элемента – вернем исходную картинку.

HTML-код:

```

```



Назначение событий

Можно из обработчика события вызывать функцию, описанную в теге **script** или отдельном файле.

Тогда, чтобы в функции было известно на каком элементе сработало событие, в качестве параметра в функцию необходимо передать ссылке на текущий элемент **this**.

Внутри обработчика события **this** ссылается на текущий элемент, то есть на тот, на котором, как говорят, «висит» (т.е. назначен) обработчик.

Назначение событий

Пример. Изменим одну картинку на другую при наведении на нее мышкой, а при снятии мыши с элемента – вернем исходную картинку.

Назначение событий

HTML-код:

```

```

Назначение событий

JavaScript-код:

```
function changePict(element, over) {
    let pict1 = 'image1.jpg';
    let pict2 = 'image2.jpg';
    element.src = (over) ? pict2 : pict1;
}
```


Назначение событий

Обработчик может быть назначен элементу, используя свойство DOM:

```
element.событие = function() {  
    операторы  
}
```

Здесь функция описана внутри обработчика событий, поэтому в ней доступны свойства текущего объекта через **this** (указатель в качестве параметра передавать не нужно).

Назначение событий

Пример. Изменим одну картинку на другую при наведении на нее мышкой, а при снятии мыши с элемента – вернем исходную картинку.

HTML-код:

```

```

Назначение событий

Пример. Изменим одну картинку на другую при наведении на нее мышкой, а при снятии мыши с элемента – вернем исходную картинку.

JavaScript-код:

```
pict.onmouseover = function() {  
    this.src = 'image2.jpg';  
};  
  
pict.onmouseout = function() {  
    this.src = 'image1.jpg';  
};
```

Назначение событий

Фундаментальный недостаток описанных выше способов назначения обработчика – невозможность повесить несколько обработчиков на одно событие.

Например, одна часть кода хочет при клике на кнопку делать её подсвеченной, а другая – выдавать сообщение.

Назначение событий

Фундаментальный недостаток описанных выше способов назначения обработчика – невозможность повесить несколько обработчиков на одно событие.

Например, одна часть кода хочет при клике на кнопку делать её подсвеченной, а другая – выдавать сообщение.

Существует альтернативный способ назначения обработчиков при помощи специального метода **addEventListener**.

Назначение событий

Синтаксис добавления обработчика:

```
element.addEventListener(event, handler, [options]);
```

- **event** - Имя события, например "click".
- **handler** - Ссылка на функцию-обработчик.
- **options** - Дополнительный объект со свойствами:

once: если true, тогда обработчик будет автоматически удалён после выполнения.

capture: фаза, на которой должен сработать обработчик.

passive: если true, то указывает, что обработчик никогда не вызовет `preventDefault()`.

Назначение событий

Существуют события, которые нельзя назначить через DOM-свойство, но можно через **addEventListener**.

Например, **DOMContentLoaded** (срабатывает, когда завершена загрузка и построение DOM документа).

```
document.onDOMContentLoaded = function() {  
    alert("DOM построен"); // не будет работать  
};  
document.addEventListener("DOMContentLoaded", function() {  
    alert("DOM построен"); // так работает  
});
```

Назначение событий

Задание 3. Гиперссылка заключена в **div**. При наведении курсора мыши на гиперссылку увеличить верхний и нижний **padding div** на 10%.

Объект event

Когда происходит событие, браузер создаёт объект события **event**, записывает в него детали и передаёт его в качестве аргумента функции-обработчику.

Некоторые свойства объекта event:

- **event.type** - тип события;
- **event.target** - элемент, на котором произошло событие.

Объект event

Пример. Изменим одну картинку на другую при наведении на нее мышкой, а при снятии мыши с элемента – вернем исходную картинку.

HTML-код:

```

```

JavaScript-код:

```
document.addEventListener("mouseover", function(event) {  
    event.target.src = 'image2.jpg';  
});  
document.addEventListener("mouseout", function(event) {  
    event.target.src = 'image1.jpg';  
});
```


События

Задание 4. При наведении курсора мыши на картинку присвоим ей класс, в котором устанавливается граница рисунка. В при снятии курсора – удаляет этот класс.

CSS-код:

```
.border {  
    border: solid thin red;  
}
```

Всплытие событий

Принцип всплытия очень простой.

Когда на элементе происходит событие, обработчики сначала срабатывают на нём, потом на его родителе, затем выше и так далее, вверх по цепочке предков.

Всплытие событий

Пример. Рассмотрим вложенную структуру тегов с обработчиком на каждом уровне.

```
<form onclick="alert('form')">
  <div onclick="alert('div')">
    <p onclick="alert('p')"></p>
  </div>
</form>
```

Клик по внутреннему **<p>** вызовет обработчик **onclick**:

1. Сначала на самом **<p>**.
2. Потом на внешнем **<div>**.
3. Затем на внешнем **<form>**.
4. И так далее вверх по цепочке до самого **document**.

Всплытие событий

Некоторые события «не всплывают».

Например, событие **focus** не всплывает..

Всплытие событий

Всегда можно узнать, на каком конкретно элементе произошло событие.

Самый глубокий элемент, который вызывает событие, называется целевым элементом, и он доступен через **`event.target`**.

Всплытие событий

Отличия **`event.currentTarget`** (**`this`**) и **`event.target`** :

- **`event.target`** – это «целевой» элемент, на котором произошло событие, в процессе всплытия он неизменен.
- **`this`** – это «текущий» элемент, до которого дошло всплытие, на нём сейчас выполняется обработчик.

Спасибо за внимание!